

MovieLens 20M en Databricks

PySpark · MinHash · LSH · FP-Growth

Camila, Marcelo, Cristian

9 de mayo de 2026

Data Mining | UTEC | 2026-I

1. **Parte I.** Preprocesamiento + EDA
2. **Parte II.** RDD vs DataFrame
3. **Parte III.** Similitud + MinHash + LSH
4. **Parte IV.** FP-Growth y reglas
5. **Parte V.** Discusión y aplicabilidad

Dataset

20 000 263 ratings

138 493 usuarios

26 744 películas

Azure Databricks 18.2 ML

Spark 4.1 + Photon

Parte I — Preprocesamiento + EDA

Limpieza

- `dropna + dropDuplicates` (defensivo)
- Normalización de géneros (Sci-Fi, Film-Noir)
- Extracción de año desde título

Binarización $\tau \geq 3.5$

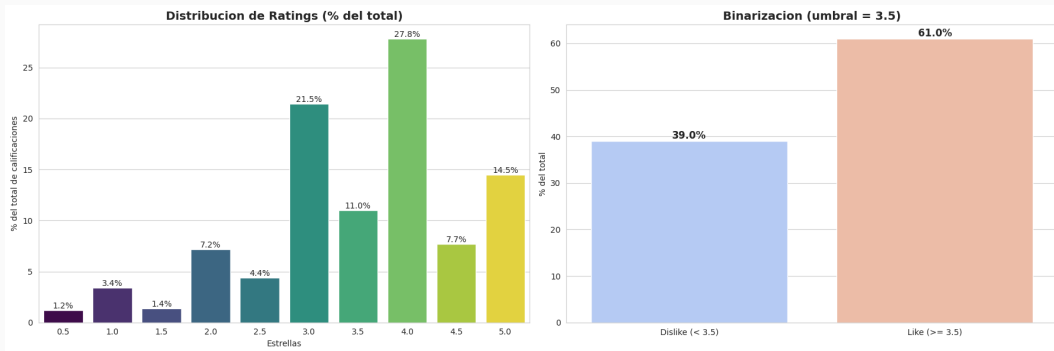
- **61 %** likes **39 %** dislikes
- Balance suficiente para señal en ambas clases

Por qué Spark aquí

`ratings.csv` pesa ~ 500 MB.

Pandas leería en $O(N)$ en un solo core; Spark lo particiona y procesa en paralelo desde el primer `read.csv`.

Distribución de ratings



Sesgo positivo: **moda en 4.0 (27.8 %)** → binarización 61/39 con $\tau=3.5$

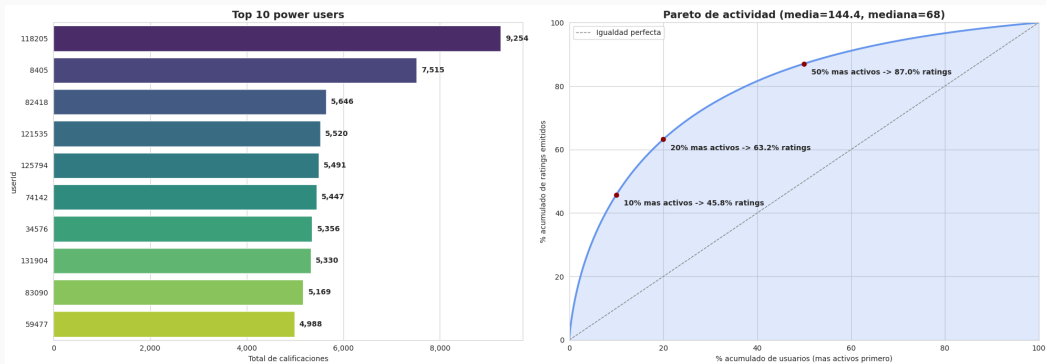
Hallazgos clave

- Sesgo positivo: **82 %** de ratings $\geq 3,0$
- Moda en 4,0 (**28 %** del total)
- Enteros acumulan 65 % pese a escala con medios puntos $\Rightarrow$ **preferencia humana por valores enteros**

Decisión: $\tau = 3,5$

- Separa el cuartil superior
- Balance **61 % / 39 %** (likes / dislikes)
- Evita clase mayoritaria que daría $\tau = 3,0$ (78 % likes) o $\tau = 4,0$ (44 % likes)

Concentración de actividad por usuario



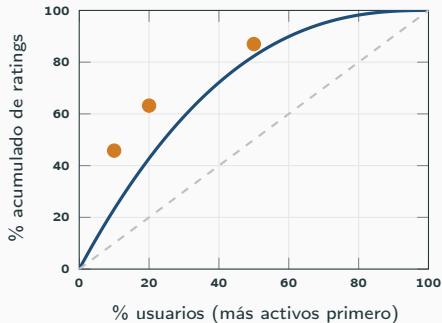
El **10 % más activo** concentra el 45.8 % de todas las calificaciones (mediana 68, máximo 9 254)

Power users dominan el dataset

- Mediana: **68** ratings
- Media: **144,4** ratings
- Máximo: **9 254** (usuario 118205)
- Ratio media/mediana: $\sim 2,1\times$

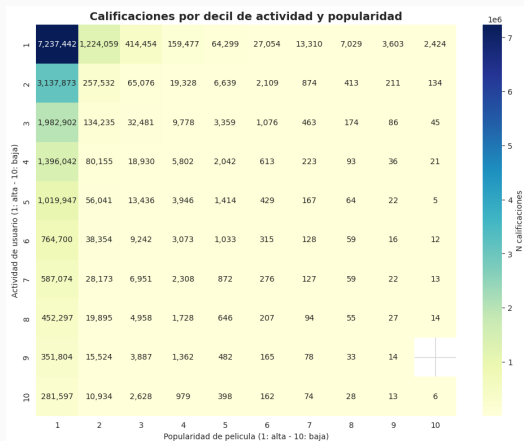
Curva de Pareto

- Top **10 %** usuarios $\rightarrow$ **45,8 %** ratings
- Top **20 %** usuarios $\rightarrow$ **63,2 %** ratings
- Top **50 %** usuarios $\rightarrow$ **87,0 %** ratings



Curva de Pareto. Diagonal punteada: igualdad perfecta.

Sparsity de la matriz usuario $\times$ película



Densidad global

0,54 %

99,46 % de celdas vacías

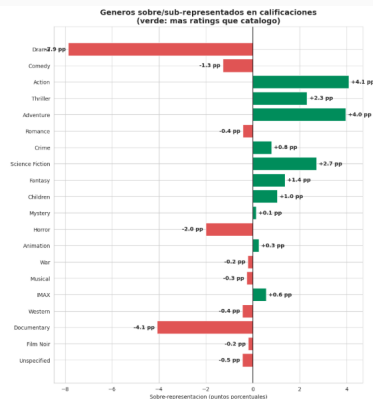
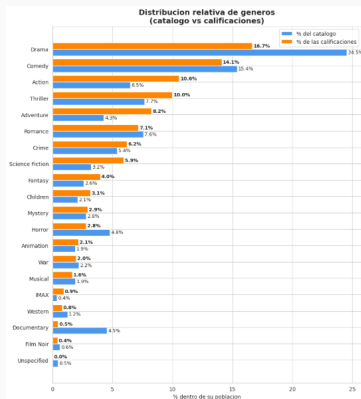
El cuadrante *top 10 % usuarios* $\times$ *top 10 % películas* concentra el **36 %** de los ratings.

→ motiva el muestreo estratificado de la Parte III.

Distribución de géneros

Por qué un gráfico relativo

Comparar conteos absolutos de películas y de ratings por género es engañoso: los totales son de órdenes de magnitud distintos. Usamos un **gráfico relativo** (% de cada género dentro de su población) para detectar géneros sobre-representados y sub-representados en consumo efectivo.



Sobre-representados *(más % ratings que catálogo)*

- **Action** (+4,1 pp): 6,5 % → **10,6 %**
- **Adventure** (+4,0 pp): 4,3 % → **8,2 %**
- Science Fiction (+2,7 pp)
- Thriller (+2,3 pp)
- Fantasy (+1,4 pp), Children (+1,0 pp)

Géneros comerciales / familiares: pocas películas acumulan muchas calificaciones por alta exposición.

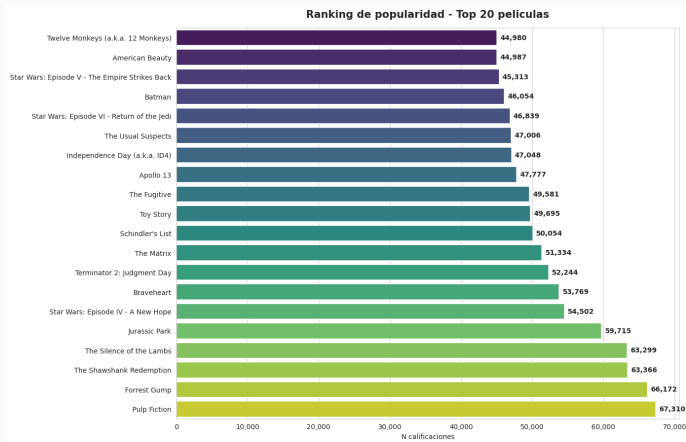
Sub-representados *(más % catálogo que ratings)*

- **Drama** (-7,9 pp): **24,5 %** catálogo → 16,7 % ratings
- **Documentary** (-4,1 pp): 4,5 % → **0,5 %**
- Horror (-2,0 pp)
- Comedy (-1,3 pp)

Catálogos de nicho: muchas películas con poca evidencia empírica de consumo.

Parte II — RDD vs DataFrame

Top-20 películas más calificadas



Pulp Fiction (67 310) *Forrest Gump* (66 172) *Shawshank* (63 366)

Top-20 películas más calificadas

#	Título	Ratings
1	Pulp Fiction	67 310
2	Forrest Gump	66 172
3	The Shawshank Redemption	63 366
4	The Silence of the Lambs	63 299
5	Jurassic Park	59 715
6	Star Wars: Episode IV	54 502
7	Braveheart	53 769
8	Terminator 2	52 244
9	The Matrix	51 334
10	Schindler's List	50 054

Hallazgos

- Top-10 dominado por **blockbusters de los 90s**. Películas que cualquier usuario activo habrá puntuado.
- Distribución **más asimétrica que la de usuarios**: media **747,8** vs mediana **18**, ratio $\sim 41\times$.
- Bottom-10: cada película con exactamente **1 rating**. Miles de películas con conteos mínimos.

Validación cualitativa

- Top-3 *moviedls* (296, 356, 318) coinciden en RDD y DataFrame $\Rightarrow$ **cómputo distribuido consistente**.

Benchmark: el mismo cómputo, dos APIs

API	Tiempo (s)	Speedup
RDD (<code>map</code> → <code>reduceByKey</code>)	31,08	1×
DataFrame (<code>groupBy</code> → <code>orderBy</code>)	1,79	17,4×

¿Por qué la diferencia?

1. **Catalyst** fusiona agregación + ordenamiento + proyección en un único plan.
2. **Tungsten** mantiene los datos en formato columnar binario en la JVM.
3. RDD paga *serialización Python* ↔ *JVM* por fila × 20 M filas.

Decisión: el resto del proyecto usa **DataFrame por defecto**.

Parte III — Similitud + MinHash + LSH

Problema

Detectar pares de películas similares (audiencias parecidas) sobre 8 320 películas.

Comparar todas-contra-todas $\rightarrow \binom{8320}{2} \approx 34,6 \text{ M pares}$.

Pipeline

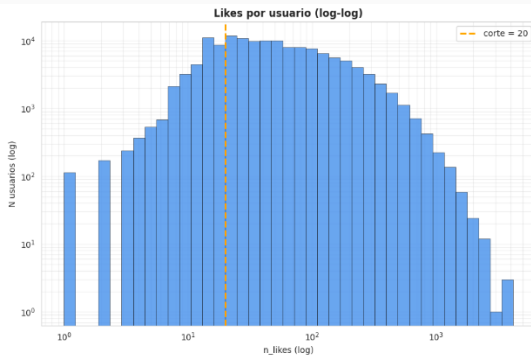
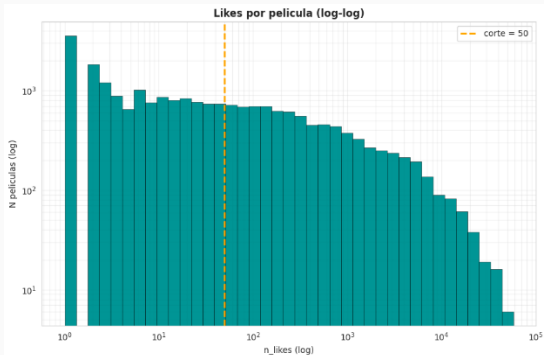
1. **Jaccard exacto** sobre muestra estratificada (507 689 pares) $\rightarrow$ *ground truth*
2. **MinHash** firma de K enteros por película
3. **LSH** reduce drásticamente los candidatos

Cada película es el *conjunto de usuarios* que le dieron like (Opción A).

Distribución de likes (justifica los cortes)

Heavy-tail extremo en ambos ejes

Histogramas log-log de likes por película y por usuario sobre el conjunto post-binarización (12,2M likes). El **corte 50/20** se aplica al final del EDA y antes de la Parte III. Total de likes (rating ≥ 3.5): 12,195,566. Películas con ≥ 1 like: 22,884. Usuarios con ≥ 1 like: 138,362



Decisión de cortes: robustez vs cobertura

Filtro	Antes	Después	% conservado
Películas con ≥ 50 likes	22 884	8 362	$\sim 36\%$
Usuarios con ≥ 20 likes	138 362	107 001	$\sim 77\%$
Cascada (ambos cortes)		8 320 películas 107 001 usuarios 11 634 899 likes	95,4 %

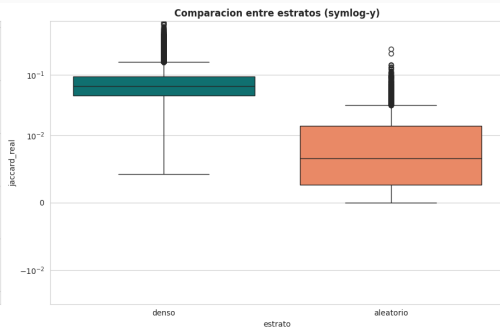
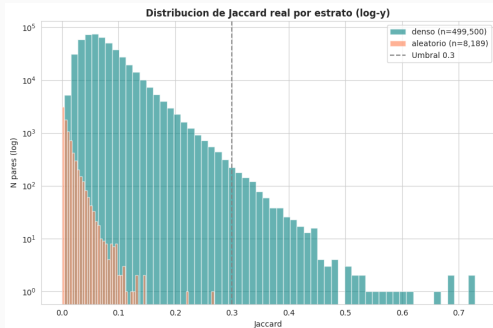
Por qué ≥ 50 likes/película

- El cuartil inferior tiene solo **3 likes** $\rightarrow$ Jaccard inestable
- Descarta el catálogo de nicho con evidencia insuficiente

Por qué ≥ 20 likes/usuario

- Usuarios con pocos likes no aportan co-preferencia informativa
- Conservamos **95,4 %** del volumen total de likes

Jaccard exacto: distribución por estrato



Estrato denso: mediana **0,065**, máximo **0,728** (LOTR Fellowship ↔ Two Towers)

Tabla 1: *

Distribución por estrato sobre **507 689** pares (499 500 densos + 8 189 aleatorios)

Estrato	n_pares	Mediana	Media	P95	Máy
Denso	499 500	0,065	0,074	0,155	0,728
Aleatorio	8 189	0,007	0,011	0,037	0,268

Top pares más similares (todos en estrato denso, todos sagas)

- LOTR Fellowship ↔ LOTR Two Towers: $J = 0,728$ (intersección 24 734 / unión 33 979)
- LOTR Two Towers ↔ LOTR Return of the King: $J = 0,724$
- Kill Bill 1 ↔ Kill Bill 2: $J = 0,683$
- Star Wars OT (4-5-6) entre sí: $J = 0,60 - 0,66$
- Harry Potter, Bourne, Indiana Jones, Padrino I/II, Alien: todos con $J > 0,55$

Compartir el 70 % de los usuarios que les dieron like es una señal robustísima de similitud temática y de audiencia.

Elección de umbrales para evaluar LSH

Umbral J	Pares positivos en GT
$\geq 0,30$	888 (888 densos + 0 aleatorios)
$\geq 0,40$	105
$\geq 0,50$	18
$\geq 0,60$	6
$\geq 0,70$	2

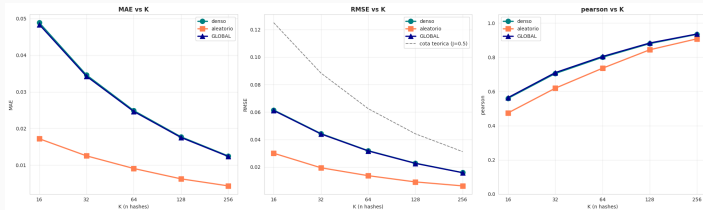
Análisis de sensibilidad

- Con umbral **0,5**: solo 18 positivos $\rightarrow$ *cualquier ruido domina* la métrica
- Con umbral **0,3**: 888 positivos $\rightarrow$ suficientes para curvas de precisión/recall robustas

Decisión

- Reportamos **ambos umbrales** como sensitivity analysis
- Si las conclusiones se sostienen para 0,3 y 0,5, son **robustas a la elección del umbral**

MinHash: ¿cuánto se pierde al aproximar?



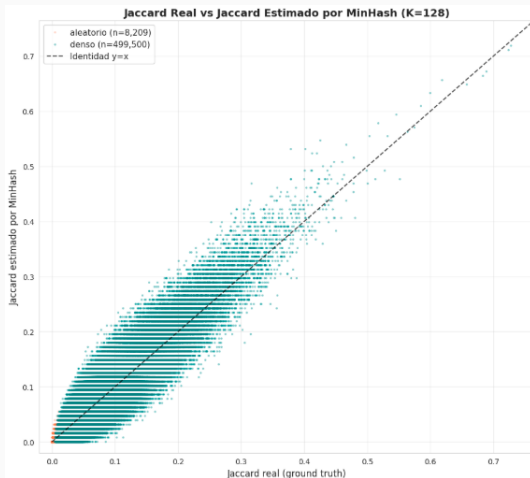
Con $K=128$ hashes:

- MAE = **0,018**
- RMSE = 0,023
- Pearson = **0,88**

Casi sin pérdida y reduce el costo por par $\sim 400\times$.

Cota teórica: el error decae como $1/\sqrt{K}$.

MinHash: validación visual del estimador



Scatter Jaccard real vs estimado con $K=128$

Los puntos se distribuyen **simétricamente alrededor de la diagonal $y = x$** sin sesgo sistemático. La nube se estrecha hacia valores altos de Jaccard (pares de sagas), donde el estimador es más preciso.

MinHash: comportamiento del error vs K

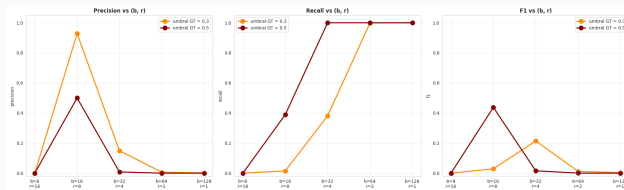
K	Estrato denso		Aleatorio	
	MAE	Pearson	MAE	Pearson
16	0,0489	0,560	0,0176	0,457
32	0,0346	0,705	0,0119	0,668
64	0,0249	0,800	0,0083	0,766
128	0,018	0,880	0,0064	0,829
256	0,0125	0,934	0,0043	0,890

Cuadruplicar K divide MAE entre $\sqrt{4} = 2$. Validación empírica de la cota $\propto 1/\sqrt{K}$.

Decisión: $K = 128$

- MAE **0,018** y Pearson **0,880**: suficiente para detectar pares similares
- Permite **5 configuraciones** (b, r) con $b \cdot r = 128$
- $K = 256$ mejora MAE en $\sim 0,005$ pero *duplica* costo, sin cambio cualitativo

LSH: barrido de bandas (b, r) con $b \cdot r = 128$



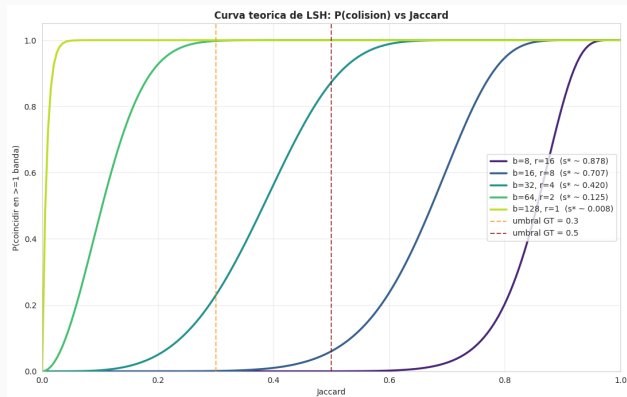
Trade-off explícito

- Más $b \Rightarrow$ más recall, menos precisión.
- Más $r \Rightarrow$ al revés.

Óptimos

- $J \geq 0,5$: $(16, 8) \rightarrow F1 = 0,438$
- $J \geq 0,3$: $(32, 4) \rightarrow F1 = 0,214$

LSH: la curva teórica predice los datos



Probabilidad de colisión

$$P(s) = 1 - (1 - s^r)^b$$

Umbral teórico:

$$s^* = (1/b)^{1/r}$$

La inflexión empírica en F1 coincide con s^* predicho por la curva.

b	r	s^*	Candidatos	F1 (0,3)	F1 (0,5)
8	16	0,878	0	0,000	0,000
16	8	0,707	19	0,029	0,438
32	4	0,420	3 545	0,214	0,016
64	2	0,125	750 105	0,012	0,000
128	1	0,008	19 810 787	0,004	0,000

(8, 16) es inviable: s^* supera el Jaccard máximo del dataset (0,728).

(64, 2) y (128, 1) requieren un *re-ranker* downstream para filtrar candidatos.

Trade-off observado en el barrido

b	r	s^*	Cand.	TP	FP	FN	P	R	F1
8	16	0,878	0	0	0	888	0,000	0,000	0,000
16	8	0,707	19	13	1	875	0,929	0,015	0,029
32	4	0,420	3 545	338	1 927	550	0,149	0,381	0,214
64	2	0,125	750 105	887	145 939	1	0,006	0,999	0,012
128	1	0,008	19 810 787	888	500 624	0	0,002	1,000	0,004

Tabla 2: *

Métricas evaluadas a $u = 0,3$ sobre los candidatos cuyo Jaccard real conocemos.

Lectura del barrido

- (8, 16) inviable: $s^* > J_{\text{máx}}$ del dataset (0,728)
- (16, 8) óptimo a $u=0,5$: **F1 = 0,44**
- (32, 4) punto dulce a $u=0,3$: **F1 = 0,21**
- (64, 2) y (128, 1): recall ~ 1 pero **99,8 % FP**

Validación con la curva teórica S

- Inflexiones empíricas coinciden con $s^* = (1/b)^{1/r}$
- (32, 4) inflexión en $s \sim 0,42 \rightarrow$ recupera $J > 0,4$
- (64, 2) inflexión en $s \sim 0,13 \rightarrow$ atrapa también $J \sim 0,15-0,2$ (FPs)

Costo del MinHash sin LSH (insumo Parte V)

Con $K = 128$: MAE **0,018** y Pearson **0,88** en estrato denso. Pérdida *despreciable*; comparar 128 enteros vs arrays de hasta 51 502 userIds (Shawshank) $\rightarrow \sim 400\times$ más barato por par.

Configuración según objetivo

- *Detección de duplicados / sagas* ($J > 0,5$): **(16, 8)** con $F1 = 0,44$
- *Recomendación exploratoria* ($J > 0,3$) sin re-ranker: **(32, 4)** con $F1 = 0,21$
- *Con re-ranker downstream*: **(64, 2)** maximiza recall (0,999) con $\sim 750K$ candidatos

Falsos positivos

- Recuperables vía re-ranker
- Algo de ruido aporta diversidad

Falsos negativos

- **Invisibles**: no se rescatan downstream
- Industria prefiere *recall alto + re-ranker* sobre *precisión alta + recall pobre*

Escalabilidad

LSH transforma

$\mathcal{O}(n^2) = 35\text{M}$ comparaciones
en $\mathcal{O}(b \cdot n) = 1\text{M}$ ops de hash
+ un self-join.

A escala Netflix ($\sim 250\text{K}$ títulos):
de ~ 30 mil millones de pares
a $\sim 32\text{M ops}$.

Reducción $\sim 1\,000\times$.

Parte IV — FP-Growth y reglas

Setup: usuarios como transacciones

Cada *transacción* es la lista de movies que un usuario calificó como *like*.

Hiperparámetros

$\text{minSupport} = 0,05$ ($\geq 5\,350$ usuarios)

$\text{minConfidence} = 0,50$

Por qué FP-Growth y no Apriori: construye un FP-tree en *una sola pasada*, sin enumerar candidatos.

Transacciones

107 001

usuarios filtrados

mediana 60 películas

máximo 3 571

6,8 s

tiempo de entrenamiento

508 K

itemsets frecuentes
($k_{\text{máx}}=9$)

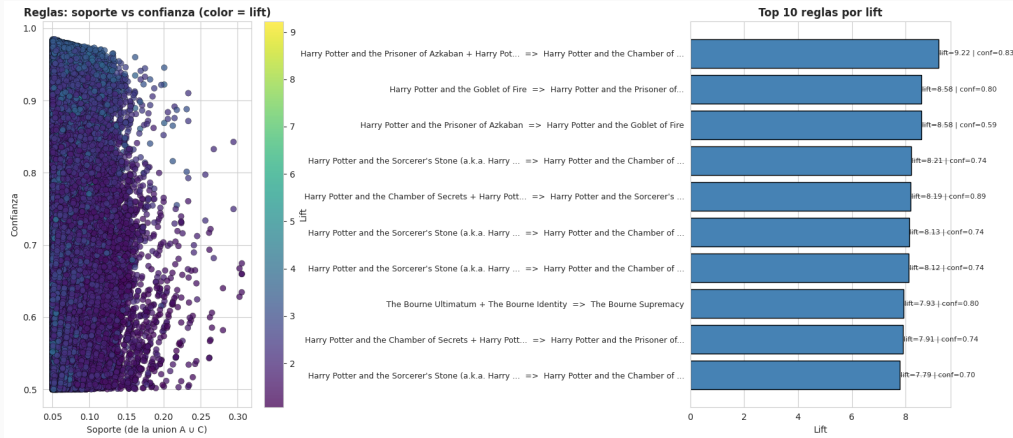
2,2 M

reglas con
confianza $\geq 0,5$

Estadísticos del conjunto de reglas

Confianza:	promedio 0,77	máximo 0,985
Lift:	promedio 2,42	máximo 9,22

Visualización del conjunto completo de reglas



Las reglas con **mayor lift** viven cerca del soporte mínimo (5 %): son *nichos informativos*, no blockbusters.

Top reglas por *lift* — las más informativas

Antecedente	Consecuente	Conf.	Lift
HP Azkaban + HP Sorcerer's Stone	HP Chamber of Secrets	0,834 1	9,219
HP Goblet of Fire	HP Prisoner of Azkaban	0,799 7	8,580
HP Sorcerer + LOTR TT + LOTR Fellowship	HP Chamber of Secrets	0,743 3	8,214
HP Chamber + HP Azkaban	HP Sorcerer's Stone	0,885 1	8,191
Bourne Ultimatum + Bourne Identity	Bourne Supremacy	0,799 9	7,929
HP Sorcerer + Shrek	HP Chamber of Secrets	0,704 9	7,790

Las primeras reglas **validan el algoritmo** (misma saga).

Las **cruzadas entre franquicias** son las que aportan valor de descubrimiento.

Confianza vs. Soporte: rankings opuestos

Top por *confianza*

Antecedentes **largos** (4–8 películas)

Consecuentes: *Godfather*, *LOTR Fellowship*

Confianza → 0,985

Lift cae a **3,08–3,41**

Sesgo: cuando el consecuente es muy popular, casi cualquier antecedente le da confianza alta.

Top por *soporte*

Antecedente y consecuente **de tamaño 1**

Pulp Fiction, Shawshank, Silence Lambs, SW

Soporte → **30,56 %**

Lift cae a 1,33–2,15

Más accionable, menos informativa: mejora apenas 37 % sobre azar.

Lift es la métrica más honesta: corrige el soporte marginal del consecuente.

589 K

reglas con *Matrix* en el antecedente

144 K

con *Matrix* en el consecuente

Matrix como objetivo (alta confianza)

Matrix Reloaded + *SW VI* + *SW V* $\Rightarrow$ *The Matrix*

confianza 0,978 · lift 2,46

Matrix como evidencia (alto lift)

HP Sorcerer's Stone + *Matrix* $\Rightarrow$ *HP Chamber*

lift 7,58

La asimetría refleja: *Matrix* es mejor *evidencia entrante* que *objetivo de recomendación*.

Parte V — Discusión Crítica

Pérdida de precisión vs. falsos positivos/negativos

Aproximar cuesta poco

MinHash ($K=128$): MAE **0,018**, Pearson **0,88**.

LSH: sí pierde, depende de (b, r) .

Ninguna config domina → definir el caso de uso primero.

FP y FN no duelen igual

FP → recuperable con re-ranker; aporta diversidad.

FN → **invisibles** para el sistema, no se arreglan downstream.

Industria: alto recall + re-ranker.

¿Spark valió la pena? — Cuellos de botella

Spark gana cuando...

- Dato distribuido $>$ RAM de un nodo
- Agregación o join nativo (`groupBy`, `array_intersect`)
- FP-Growth nativo

Spark pierde cuando...

- UDFs Python por fila
- Resultado intermedio cabe en RAM

Pandas fue $\sim 5\times$ más rápido para postprocesar reglas.

Tres dimensiones que escalan mal

Memoria matriz densa $3,7 \times 10^9$ celdas $\rightarrow$ inviable.

Tiempo RDD escala mal por serialización.

Cardinalidad cross-join naive $\rightarrow$ cuadrático.

“Agregar en Spark, presentar en pandas.”

Netflix

~250 K títulos, 230 M usuarios.

Naive: $3,1 \times 10^{10}$ pares.

Con LSH: $\sim 32 \times 10^6$.

~1 000× de mejora.

Spotify

~100 M canciones.

Cambia Jaccard por **embeddings**.

Cambia LSH por **ANN**: FAISS,
ScaNN, HNSW.

Mismo esquema arquitectural.

Amazon

Reglas FP-Growth como base de
"los clientes que compraron X
también compraron Y".

minSupport inversamente
proporcional al catálogo.

Paradigma de producción: **retrieval barato + ranking caro**.

Nuestro proyecto reproduce el primer eslabón.

Conclusión

17×

DataFrame vs RDD

(Parte II)

400×

MinHash vs comparación
exacta de conjuntos

(Parte III)

1 000×

LSH a escala industrial

(Netflix-like)

Cierre

La arquitectura *retrieval barato + ranking caro* es reproducible con estas herramientas. El escalado a producción es cuestión de **paralelismo** y de **reemplazar conjuntos por embeddings**, pero el esqueleto algorítmico es el mismo.

Gracias

¿Preguntas?

MovieLens 20M · Databricks · PySpark / MinHash / LSH / FP-Growth